Corretude de Algoritmos e Indução Matemática

Da prova formal de que um algoritmo funciona à análise rigorosa de sua complexidade

Felipe A. Przysiada

Como saber que um algoritmo funciona?

Historicamente, duas tradições respondem a essa pergunta: a retórica e persuasão (Grécia antiga, ciências não-exatas) e o método científico.

Método científico	O que é
Empírico: Testes	Executar o algoritmo com amostras de instâncias e observar o resultado
Teórico: Prova de corretude	Demonstrar matematicamente que o algoritmo sempre produz o resultado certo

As limitações de cada abordagem

- Testes talvez não encontrem bugs obscuros, casos-limite raros passam despercebidos.
- Usar somente testes pode ser perigoso em sistemas críticos.
- Provas de corretude também podem conter bugs, uma prova errada passa despercebida também.

Conclusão

O melhor é usar uma combinação de testes e prova de corretude, cada um cobre pontos cegos do outro.

Quando só testar não bastou: bugs reais e caros

Caso real	O que aconteceu	Causa
Ariane 5 (1996)	Foguete explodiu 37s após o lançamento, US$ 370 milhões perdidos	Overflow numérico não coberto pelos testes do software reaproveitado do Ariane 4
Therac-25 (1985-87)	Máquina de radioterapia aplicou doses letais de radiação	Condição de corrida (race condition) entre threads, nunca reproduzida em teste
Knight Capital (2012)	US$ 440 milhões perdidos em 45 minutos de pregão	Código morto reativado por engano; nenhum teste cobria esse caminho
Patriot Missile (1991)	Falha ao interceptar míssil, 28 mortes	Erro de arredondamento acumulado em contagem de tempo, não detectado em testes curtos

O padrão em comum

Em todos os casos, os testes cobriram os cenários 'esperados', mas não provaram a corretude para TODO n ou TODA entrada possível. É exatamente essa garantia universal ("para todo n") que uma prova de corretude por indução oferece, e que nenhuma bateria finita de testes pode dar.

Para provar a corretude de um algoritmo recursivo

1. Provar por indução no tamanho da entrada.

2. A base da recursão é a base da indução.

3. Mostrar que as chamadas recursivas sempre são para instâncias menores (geralmente trivial).

4. Passo indutivo: assumir que as chamadas recursivas funcionam corretamente, e usar essa suposição para provar que a chamada atual funciona corretamente.

Ideia central

É a MESMA lógica da indução matemática: base ↔ caso base da recursão; hipótese ↔ 'confio que a recursão já funciona para entradas menores'; passo ↔ compor a solução.

Fibonacci recursivo

$$F_0 = n \qquad \text{se } n \leq 1$$
$$F_n = F_{n-1} + F_{n-2} \qquad \text{se } n > 1$$

```
Algoritmo fib(n)
  se n ≤ 1 então retorna n
  senão retorna fib(n-1) + fib(n-2)
```

Teorema. Para todo $n \geq 0$, fib(n) devolve F_n.

Observação

A chamada fib(n) depende de DUAS chamadas anteriores (n-1 e n-2), não apenas 'a anterior'.
Isso é o sinal de que a prova exige indução FORTE, não fraca.

Aplicação real

Sequências recorrentes como Fibonacci modelam crescimento populacional idealizado, o número de espirais em girassóis e conchas, e são a base de estruturas de dados reais como a Fibonacci heap (usada em implementações eficientes do algoritmo de Dijkstra).

Prova: fib(n) devolve F□

Base:

Para n = 0, fib(n) devolve 0 como afirmado. Para n = 1, fib(n) devolve 1 como afirmado.

Hipótese (indução forte):

Para n ≥ 2 e para todo 0 ≤ m < n, fib(m) devolve F□.

Passo: queremos mostrar que fib(n) devolve F□:

O que fib(n) devolve?

fib(n-1) + fib(n-2) = $F_{□-1} + F_{□-2}$ (hipótese de indução, pois n-1 < n e n-2 < n)

= F□ (definição de Fibonacci).

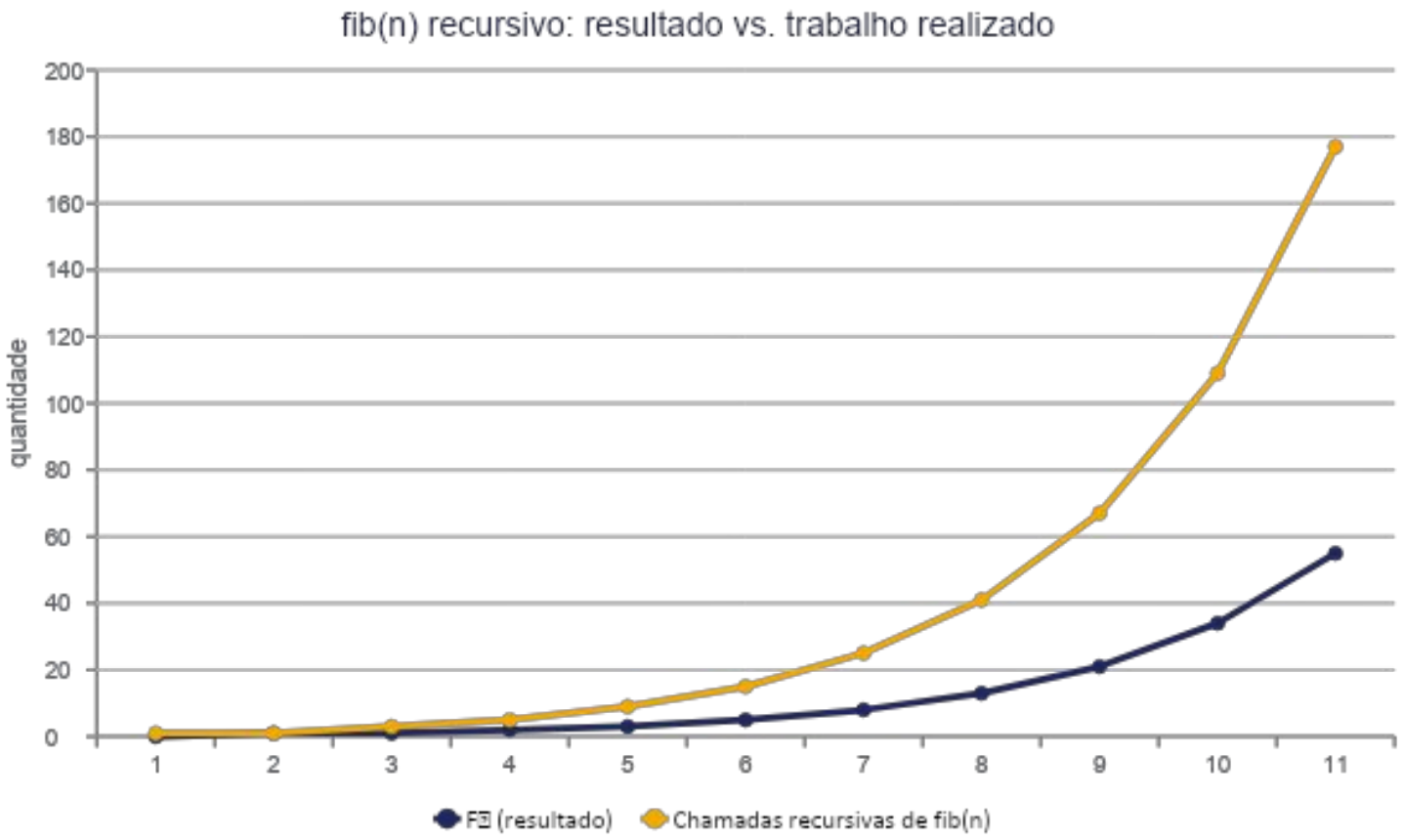
Ponto crítico da prova forte

Precisamos que AMBOS n-1 e n-2 sejam menores que n e maiores ou iguais à base, por isso a hipótese cobre 'todo 0 ≤ m < n', não apenas m = n-1.

Isso prova a corretude, mas não diz nada sobre eficiência

fib(n) recursivo está PROVADAMENTE correto, mas, como veremos no gráfico a seguir, extremamente ineficiente.

Correto não é o mesmo que eficiente



O que o gráfico revela

O número de chamadas recursivas cresce muito mais rápido que o próprio resultado F□, de forma exponencial (≈ 2·F□₊₁ - 1).

Impacto real

Para n = 40, o fib recursivo ingênuo já faz mais de 300 milhões de chamadas. Isso motiva a versão ITERATIVA que provamos corretude a seguir, mesmo resultado, ordem de grandeza menor de trabalho.

EXERCÍCIO · CORRETUDE 1

Prove a corretude de maximo(A[1..n])

```
Algoritmo maximo(A[1..n])
  se n ≤ 1 então retorna A[1]
  senão retorna max(maximo(A[1..n-1]), A[n])
```

Enunciado

Prove que, para todo n ≥ 1, maximo(A[1..n]) devolve max{A[1], A[2], ..., A[n]}.

Dica

Aqui a chamada recursiva depende SÓ do caso anterior (n-1), isso é indução fraca. Separe o último termo A[n] e use a hipótese sobre maximo(A[1..n-1]).

maximo(A[1..n]) devolve max{A[1],...,A[n]}

Base (n=1):
maximo(A[1..n]) devolve A[1], como afirmado.

Hipótese (n ≥ 1):
maximo(A[1..n]) devolve max{A[1], A[2], ..., A[n]}.

Passo: mostrar que maximo(A[1..n+1]) devolve max{A[1],...,A[n+1]}:
O que maximo(A[1..n+1]) devolve?
max(maximo(A[1..n]), A[n+1])
= max(max{A[1],...,A[n]}, A[n+1]) (hipótese de indução)
= max{A[1], A[2], ..., A[n+1]}.

Compare com fib(n)
Aqui bastou olhar 'um passo atrás' (n → n+1). Em fib, precisamos olhar dois passos atrás, daí a necessidade de indução forte.

Generalização
Esse é o esqueleto de toda prova recursiva 'linear': base trivial, hipótese sobre n, passo que separa o último elemento processado.

Busca binária recursiva

```
Algoritmo buscaBin(A[1..n], alvo, ini, fim)
  se ini > fim então retorna "não encontrado"
  meio ← ⌊(ini+fim)/2⌋
  se A[meio] = alvo então retorna meio
  senão se A[meio] > alvo então
    retorna buscaBin(A, alvo, ini, meio-1)
  senão
    retorna buscaBin(A, alvo, meio+1, fim)
```

Por que a prova é forte

O tamanho do subvetor (fim-ini) diminui a cada chamada, mas não por 1 fixo, a hipótese deve cobrir TODO subintervalo menor, não apenas 'o anterior'.

Esboço da prova (indução no tamanho do intervalo, k = fim - ini + 1)

Base: $k \leq 0$ (ini > fim) $\longrightarrow$ devolve corretamente 'não encontrado'.

Passo: A[meio] resolve o caso 'achou'; senão, a chamada recursiva atua sobre um intervalo estritamente menor, e por hipótese está correta.

Conexão com a aula anterior

Esse mesmo algoritmo é nosso exemplo predileto para $T(n) = T(n/2) + O(1)$ e complexidade.
$O(\log n)$, a prova de corretude e a análise de complexidade caminham juntas.

Para provar a corretude de um algoritmo iterativo

1. Identificar laços: analisar um laço por vez, começando do mais interno.
2. Achar invariantes: para cada laço, achar um invariante de laço que permanece verdade em toda repetição e captura o 'progresso' feito pelo laço.
3. Provar invariantes: provar que os invariantes de laço são verdadeiros.
4. Provar término: usar os invariantes de laço para provar que o algoritmo termina.
5. Provar corretude: usar os invariantes de laço para provar que o algoritmo obtém o resultado correto.

A parte mais difícil

Achar o invariante certo é, quase sempre, o passo mais difícil, ele precisa ser forte o bastante para implicar corretude, mas simples o bastante para provar por indução.

Notação: o valor de uma variável após i iterações

O valor da variável x imediatamente após a i-ésima iteração do laço é denotado por x_i
(i = 0 significa imediatamente antes de entrar no laço pela primeira vez)

Exemplo

x_6 é o valor da variável x depois da sexta vez que o laço foi executado. Vamos nos concentrar em algoritmos de um único laço.

Por que essa notação existe

Um invariante de laço é uma afirmação sobre x_i que vale para TODO i, ou seja, é uma propriedade P(i) que provamos por indução em i, exatamente como fizemos com P(n).

Ponte com a recursão

Isso significa que TODA prova de invariante de laço É uma prova por indução, só muda o nome da variável que indexamos (i em vez de n).

Fibonacci iterativo

```
Algoritmo fib(n)
  se n = 0 então retorna 0
  a ← 0; b ← 1; i ← 2
  enquanto i ≤ n faça
    c ← a+b; a ← b; b ← c; i ← i+1
  retorna b
```

Fatos sobre o algoritmo (recorrências auxiliares)

$i_0=2$, $i_{j+1}=i_j+1$

$a_0=0$, $a_{j+1}=b_j$

$b_0=1$, $b_{j+1}=c_{j+1}=a_j+b_j$

Invariante de laço. Para todo natural $j \geq 0$, $i_j = j+2$, $a_j = F_j$ e $b_j = F_{j+1}$.

Impacto real (compare com o gráfico do slide 7)

Essa versão faz exatamente n-1 iterações, trabalho LINEAR, contra o crescimento exponencial da versão recursiva ingênua.

O que vem agora

Vamos provar o invariante por indução em j, depois usá-lo para provar término e corretude, o mesmo roteiro de 3 etapas do slide 11.

Prova do invariante de laço (indução em j)

Base (j=0):

trivial, pois $i_0=2$, $a_0=0=F_0$ e $b_0=1=F_1$.

Hipótese:

Para $j \geq 0$, $i_j=j+2$, $a_j=F_j$ e $b_j=F_{j+1}$.

Passo: queremos mostrar que $i_{j+1}=j+3$, $a_{j+1}=F_{j+1}$ e $b_{j+1}=F_{j+2}$:

$i_{j+1} = i_j + 1$ (hip.) $= (j+2)+1 = j+3$.

$a_{j+1} = b_j$ (hip.) $= F_{j+1}$.

$b_{j+1} = c_{j+1} = a_j + b_j$ (hip.) $= F_j + F_{j+1} = F_{j+2}$.

Compare com a parte anterior

Essa é, literalmente, uma prova por indução fraca em j, base, hipótese em j, passo para j+1.

O 'algoritmo' e a 'matemática pura' usam a mesma ferramenta lógica.

Prova de término e corretude

Teorema.
O algoritmo termina com b contendo F□.

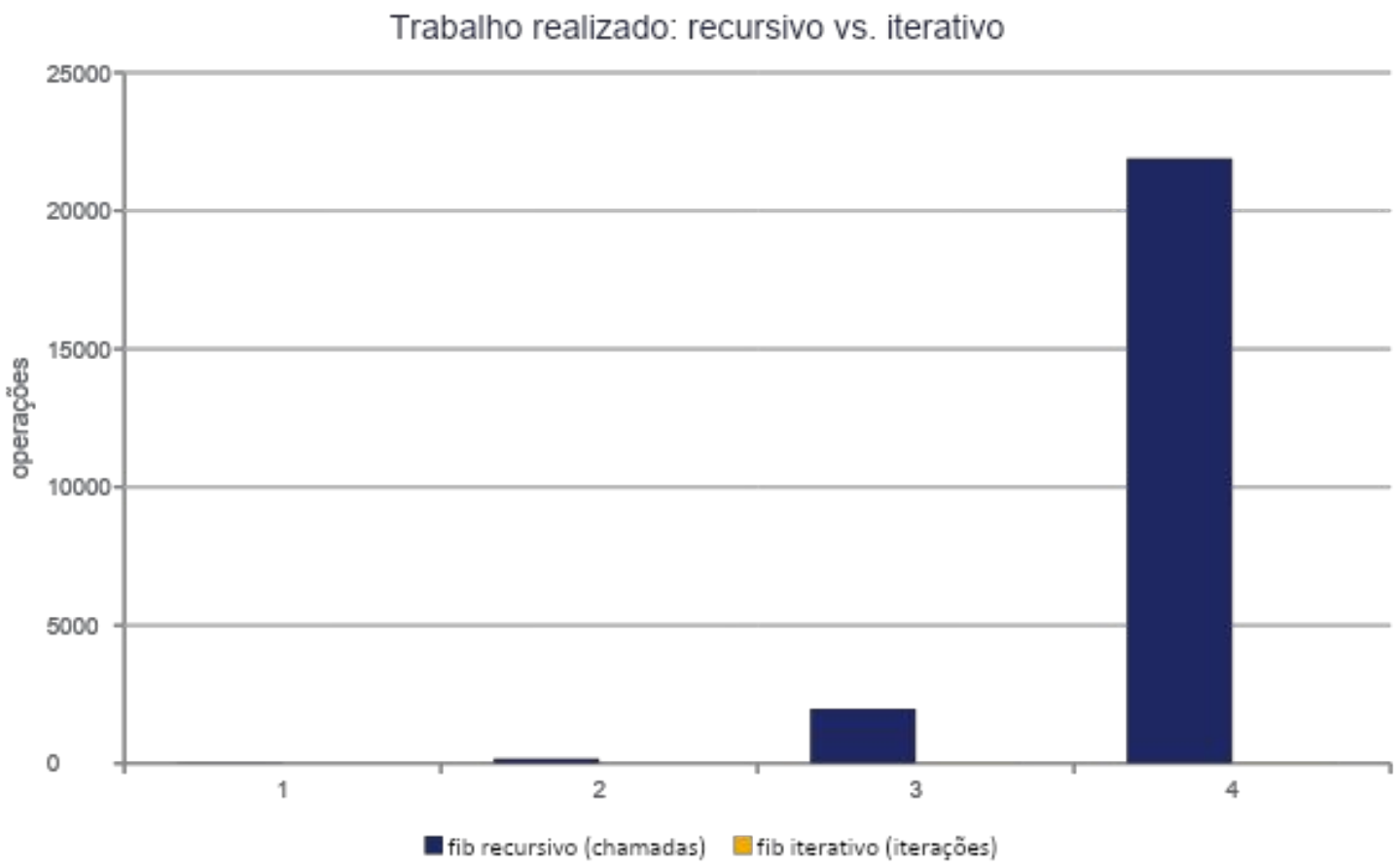
Término
A afirmação é claramente verdadeira se n=0. Se n>0, entramos no laço. Como i_{j+1}=i_j+1, eventualmente i será igual a n+1 e o laço termina. Suponha que isso acontece após t iterações:
como i_t=n+1 e i_t=t+2, conclui-se que t=n-1.

Corretude
Pelo invariante de laço, b_t = F_{t+1} = F_n.

O padrão geral
Termino: invariante mostra que uma variável de controle caminha monotonamente até um valor-limite. Corretude: no ponto de parada, o invariante 'colapsa' na propriedade desejada.

Agora vamos ver um segundo exemplo iterativo, multiplica(y,z), com um invariante mais elaborado.

O custo de duas versões corretas do mesmo problema



Leitura do gráfico

Em n=20, a versão recursiva já faz ~22 mil chamadas contra ~19 iterações da versão iterativa, a barra iterativa é quase invisível na mesma escala.

A lição desta seção

Ambos os algoritmos são igualmente CORRETOS (provamos os dois!). A escolha entre eles é uma questão de complexidade.

Multiplicação russa camponesa

```
Algoritmo multiplica(y, z)
  x ← 0
  enquanto z > 0 faça
    se z é ímpar então x ← x + y
    y ← 2y
    z ← ⌊z/2⌋
  retorna x
```

Contexto histórico e real

Técnica usada por camponeses russos e egípcios antigos para multiplicar sem tabuada, e é exatamente o algoritmo implementado em hardware de multiplicadores binários de CPUs modernas (deslocamentos de bit em vez de multiplicação direta).

Invariante de laço. Para todo natural $j \geq 0$, $y_j \cdot z_j + x_j = y_0 \cdot z_0$.

Ideia intuitiva do invariante

$y_j \cdot z_j$ é 'o que ainda falta multiplicar' e x_j é 'o que já foi acumulado', a soma dos dois nunca muda, embora cada peça mude a cada iteração.

Complexidade

Como z é dividido por 2 a cada iteração, o laço executa $\Theta(\log z)$ vezes, vamos provar esse limite formalmente por indução mais adiante.

Prova completa: invariante, término e corretude

Base (j=0):

trivial, pois $y_0 z_0 + x_0 = y_0 z_0 + x_0 = y_0 z_0$ (como $x_0 = 0$).

Passo: mostrar que $y_{j+1} z_{j+1} + x_{j+1} = y_0 z_0$:

$$y_{j+1} z_{j+1} + x_{j+1} = 2y_j \cdot \lfloor z_j / 2 \rfloor + x_j + y_j \cdot (z_j \bmod 2)$$
$$= y_j \cdot (2\lfloor z_j / 2 \rfloor + (z_j \bmod 2)) + x_j = y_j z_j + x_j = y_0 z_0.$$

Término

A cada iteração z é dividido pela metade (arredondado para baixo). Logo, para alguma iteração t, $z_t = 0$ e o laço termina.

Corretude

Pelo invariante, $y_t \cdot z_t + x_t = y_0 z_0$. Como $z_t = 0$, temos $x_t = y_0 z_0 = yz$.

Resultado preliminar usado

A prova usa o fato $2\lfloor z/2 \rfloor + (z \bmod 2) = z$, também provado por casos (z par / z ímpar), um pequeno lema auxiliar comum em provas com divisão inteira.

EXERCÍCIO · CORRETUDE 2

Prove a corretude de insertionSort

```
Algoritmo insertionSort(A[1..n])
  para j de 2 até n faça
    chave ← A[j]
    i ← j-1
    enquanto i > 0 e A[i] > chave faça
      A[i+1] ← A[i]; i ← i-1
    A[i+1] ← chave
```

Enunciado

Considere o invariante do laço externo: 'no início de cada iteração j, o subvetor A[1..j-1] está ordenado'. Use-o para esboçar (base, passo, término, corretude) a prova de que insertionSort deixa A[1..n] ordenado.

Dica

Termino é imediato (j vai de 2 até n, incremento fixo). O ponto central é o passo: por que inserir 'chave' na posição certa de A[1..j-1] ordenado preserva o invariante para j+1?

insertionSort ordena A[1..n]

Invariante:
No início de cada iteração j do laço externo, A[1..j-1] está ordenado.

Base (j=2):
A[1..1] tem um único elemento, logo está trivialmente ordenado.

Passo: assumindo A[1..j-1] ordenado, mostrar que A[1..j] fica ordenado ao final da iteração j:
O laço interno desloca para a direita todo elemento de A[1..j-1] maior que 'chave' = A[j], e insere 'chave' na primeira posição i+1 onde A[i] ≤ chave. Como A[1..j-1] já estava ordenado, o resultado A[1..j] também fica ordenado.

Término
O laço externo executa exatamente n-1 vezes (j de 2 até n); o laço interno sempre decresce i, terminando quando i=0 ou A[i]≤chave.

Corretude
Ao terminar (j=n+1), o invariante garante que A[1..n] está ordenado.

Guardem este algoritmo: voltaremos a ele para contar exatamente o número de comparações.

PONTE

A mesma ferramenta lógica, usada três vezes

Agora vamos formalizar a indução em si e ganhar duas ferramentas novas: somatório e soma telescópica

Princípio da indução matemática (forma fraca)

Seja $P(n)$ uma propriedade sobre os naturais $n \geq n_0$. Se
(i) $P(n_0)$ é verdadeira (base) e (ii) para todo $k \geq n_0$, $P(k) \Longrightarrow P(k+1)$ (passo)
então $P(n)$ é verdadeira para todo $n \geq n_0$.

Intuição das peças de dominó

A base derruba a primeira peça; o passo garante que, se uma peça cai, a próxima também cai, logo todas caem a partir de n_0.

Erro comum

Provar só a base, ou só o passo, não prova nada, sem a base, a cadeia do passo nunca começa a valer de fato.

Já usamos esse esquema três vezes nesta aula

Corretude de fib(n) recursivo, do invariante de multiplica(y,z), e do invariante de fib(n) iterativo, cada um é uma instância de $P(n_0)$, passo, conclusão.

Indução fraca vs. indução forte

	Indução fraca	Indução forte
Hipótese	P(n) vale	P(m) vale para TODO $n_0 \leq m < n$
Passo usa	apenas o caso anterior	qualquer caso menor já provado
Já vimos hoje	maximo(A[1..n]) e insertionSort	fib(n) recursivo e busca binária
Quando escolher	recorrência de 1 passo: T(n) usa T(n-1)	recorrência de vários/variados passos: T(n) usa T(n-1) e T(n-2), ou T(n/2)

Equivalência

As duas formas são logicamente equivalentes, a forte é apenas mais conveniente quando o passo precisa 'olhar para trás' mais de uma posição.

Dica prática

Olhe a recorrência do algoritmo: se referencia só o passo anterior, indução fraca basta; se referencia dois ou mais, use indução forte.

EXEMPLO RESOLVIDO

A soma de Gauss: 1 + 2 + … + n = n(n+1)/2

Contexto histórico

Diz a lenda que Gauss, aos 9 anos, somou 1..100 instantaneamente pareando extremos. Vamos provar isso por indução.

Teorema. Para todo $n \geq 1$, $\sum_{i=1}^{n} i = n(n+1)/2$.

Base (n=1):

$\sum_{i=1}^{1} i = 1$ e $1 \cdot 2/2 = 1$. Vale.

Hipótese ($n \geq 1$):

$\sum_{i=1}^{n} i = n(n+1)/2$.

Passo: queremos mostrar que $\sum_{i=1}^{n+1} i = (n+1)(n+2)/2$:

$\sum_{i=1}^{n+1} i = (\sum_{i=1}^{n} i) + (n+1)$ (separando o último termo)

$= n(n+1)/2 + (n+1)$ (hipótese de indução)

$= (n+1)(n+2)/2$.

Padrão a reter

Separar o último termo do somatório é o movimento-chave em quase toda prova indutiva envolvendo Σ.

EXERCÍCIO · BLOCO 1

Prove por indução: $\Sigma^{n}_{i=1}\, i^2 = n(n+1)(2n+1)/6$

Enunciado

Prove, para todo $n \geq 1$, que $1^2 + 2^2 + \ldots + n^2 = n(n+1)(2n+1)/6$, usando base, hipótese e passo (separando o último termo).

Passo a passo sugerido

1. Verifique a base para n=1. 2. Escreva a hipótese para n. 3. No passo, separe $(n+1)^2$ e substitua a soma pela hipótese.

Por que essa fórmula importa

Σi^2 aparece ao contar o custo de algoritmos com laços cujo trabalho interno cresce linearmente com o índice, por exemplo, ao processar cada linha i de uma matriz n×n com i operações.

$\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6$

Base (n=1):

$1^2 = 1$ e $1 \cdot 2 \cdot 3/6 = 1$. Vale.

Hipótese ($n \geq 1$):

$\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6$.

Passo: mostrar que $\sum_{i=1}^{n+1} i^2 = (n+1)(n+2)(2n+3)/6$:

$$\sum_{i=1}^{n+1} i^2 = n(n+1)(2n+1)/6 + (n+1)^2 \quad \text{(hipótese + último termo)}$$

$$= (n+1)\cdot[\, n(2n+1) + 6(n+1) \,] / 6$$

$$= (n+1)(2n^2+7n+6)/6 = (n+1)(n+2)(2n+3)/6.$$

Pegadinha comum

Confira a fatoração $2n^2+7n+6 = (n+2)(2n+3)$ por distributiva antes de seguir.

Padrão a reter

A técnica é idêntica à do slide anterior: só muda a álgebra final.

Exemplo real: Torres de Hanói

```
Algoritmo hanoi(n, origem, aux, destino)
  se n = 0 então retorna
  hanoi(n-1, origem, destino, aux)
  mover disco n de origem para destino
  hanoi(n-1, aux, origem, destino)
```

Recorrência de T(n) = número de movimentos

$T(0) = 0$

$T(n) = 2 \cdot T(n-1) + 1$

Conjectura: $T(n) = 2^n - 1$.

Base (n=0):

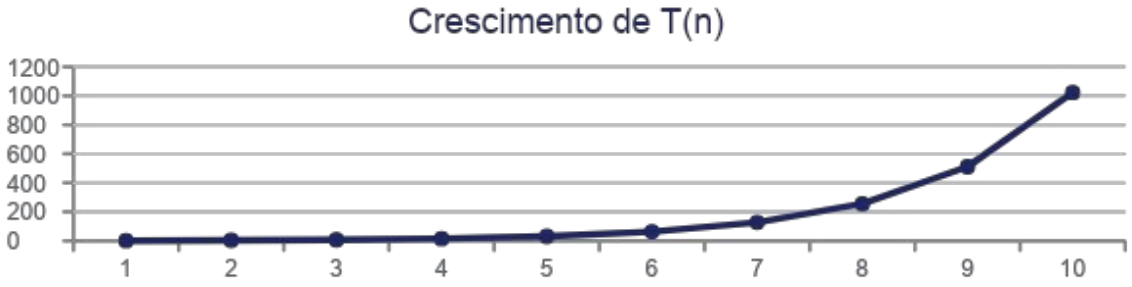
$T(0) = 0 = 2^0 - 1$. Vale.

Hipótese ($n \geq 0$):

$T(n) = 2^n - 1$.

Passo: mostrar que $T(n+1) = 2^{n+1} - 1$:

$T(n+1) = 2 \cdot T(n) + 1 = 2 \cdot (2^n - 1) + 1 = 2^{n+1} - 1$.



Impacto real

Para n=64 (lenda do templo), $2^{64} - 1$ movimentos a 1/segundo levariam ~585 bilhões de anos, a indução prova o limite EXATO, não uma estimativa.

EXERCÍCIO · BLOCO 2

Quantas iterações multiplica(y,z) executa?

Enunciado

Retomando multiplica(y,z) da Parte I: seja C(z) o número de iterações do laço para $z \geq 1$.
Prove por indução FORTE (sobre z) que $C(z) = \lfloor \log_2 z \rfloor + 1$.

Dica

Cada iteração reduz z para $\lfloor z/2 \rfloor$. Logo $C(z) = 1 + C(\lfloor z/2 \rfloor)$ para $z \geq 1$, com $C(0) = 0$.
Use $\lfloor \log_2 \lfloor z/2 \rfloor \rfloor = \lfloor \log_2 z \rfloor - 1$ (para $z \geq 1$).

Por que importa

Esse é o argumento que sustenta a complexidade O(log z) da multiplicação russa camponesa, a mesma ideia usada em exponenciação rápida e em algoritmos de criptografia RSA (exponenciação modular), onde reduzir o número de multiplicações de z para log z é a diferença entre 'instantâneo' e 'impraticável'.

$C(z) = \lfloor \log_2 z \rfloor + 1$

Base (z=1):

C(1) = 1+C(0) = 1 e $\lfloor \log_2 1 \rfloor$+1 = 1. Vale.

Hipótese (indução forte):

Para todo $1 \leq q < z$, $C(q) = \lfloor \log_2 q \rfloor + 1$.

Passo: para $z \geq 2$, mostrar que $C(z) = \lfloor \log_2 z \rfloor + 1$:

C(z) = 1 + C($\lfloor z/2 \rfloor$) (definição da recorrência)

= 1 + ($\lfloor \log_2 \lfloor z/2 \rfloor \rfloor$ + 1) (hipótese, pois $\lfloor z/2 \rfloor < z$)

= 1 + ($\lfloor \log_2 z \rfloor$ - 1 + 1) = $\lfloor \log_2 z \rfloor$ + 1.

Erro comum

Usar indução fraca aqui falha: $\lfloor z/2 \rfloor$ não é z-1, então a hipótese sobre 'o caso anterior' não cobre o que precisamos.

Conclusão: multiplica(y,z) executa $\Theta(\log z)$ iterações, a mesma ordem de crescimento da busca binária.

Definição formal do somatório

$$\sum_{i=\square}^{n} a_i = a_{\square} + a_{\square+1} + \ldots + a_{\square} \qquad (m \leq n)$$

Definição recursiva (a mesma ideia de indução aparece aqui!)

Caso base (soma vazia): $\sum_{i=\square}^{m-1} a_i = 0$

Passo: $\sum_{i=\square}^{n} a_i = \sum_{i=\square}^{n-1} a_i + a_{\square}$ (para $n \geq m$)

Vocabulário

i é o índice (variável muda); m é o limite inferior; n é o limite superior; a_i é o termo geral. Trocar i por k não muda o valor da soma.

Ligação com algoritmos

Quando 'para i de m até n faça ... custo(i) ...' executa, o custo total é exatamente $\sum_{i=\square}^{n}$ custo(i).

Já usamos isso implicitamente: multiplica(y,z) tem $\Theta(\log z)$ iterações porque contamos exatamente esse somatório disfarçado de recorrência.

Propriedades algébricas do somatório

Propriedade	Enunciado
Linearidade	$\sum_{i=□}^{n} (a_i + b_i) = \sum_{i=□}^{n} a_i + \sum_{i=□}^{n} b_i$
Fator constante	$\sum_{i=□}^{n} (c \cdot a_i) = c \cdot \sum_{i=□}^{n} a_i$ (c não depende de i)
Separação de intervalo	$\sum_{i=□}^{n} a_i = \sum_{i=□}^{k} a_i + \sum_{i=□+1}^{n} a_i$ $(m \leq k < n)$
Mudança de variável	$\sum_{i=□}^{n} a_i = \sum_{□=0}^{n-m} a_{□+□}$ (substituindo j = i - m)
Soma constante	$\sum_{i=□}^{n} c = (n - m + 1) \cdot c$

Erro comum

$\sum_{i=□}^{n} (a_i \cdot b_i) \neq (\sum a_i) \cdot (\sum b_i)$ em geral, o produto NÃO distribui sobre o somatório como a soma faz.

Uso típico

Separação de intervalo é o truque usado para 'isolar' o último termo nas provas por indução que fizemos.

EXERCÍCIO · BLOCO 3

Manipule o somatório

Enunciado

Usando as propriedades do slide anterior, simplifique $\Sigma^{n}_{i=1}$ (3i + 2) em função de n, sem somar termo a termo.

Dica

Separe com linearidade: Σ3i e Σ2. Aplique fator constante no primeiro e soma constante no segundo, depois use Gauss para Σi.

Por que praticar isso

Essa é a manipulação que você faz ao calcular o número exato de operações de um algoritmo cujo laço soma um custo que cresce linearmente com o índice, por exemplo, ao somar o número de trocas em insertionSort, onde cada iteração j pode fazer até j-1 deslocamentos mais 2 operações fixas de controle de laço.

$\Sigma^{n}_{i=1}\,(3i + 2) = (3n^2 + 7n)/2$

Resolução

$\Sigma^{n}_{i=1}\,(3i + 2)$ $= \Sigma^{n}_{i=1}\,3i \;+\; \Sigma^{n}_{i=1}\,2$ (linearidade)

$= 3\cdot\Sigma^{n}_{i=1}\,i \;+\; 2\cdot\Sigma^{n}_{i=1}\,1$ (fator constante)

$= 3\cdot[\, n(n+1)/2 \,] \;+\; 2\cdot n$ (Gauss + soma constante)

$= 3n(n+1)/2 + 2n \;=\; (3n^2 + 7n)/2.$

Verificação rápida (n=2)

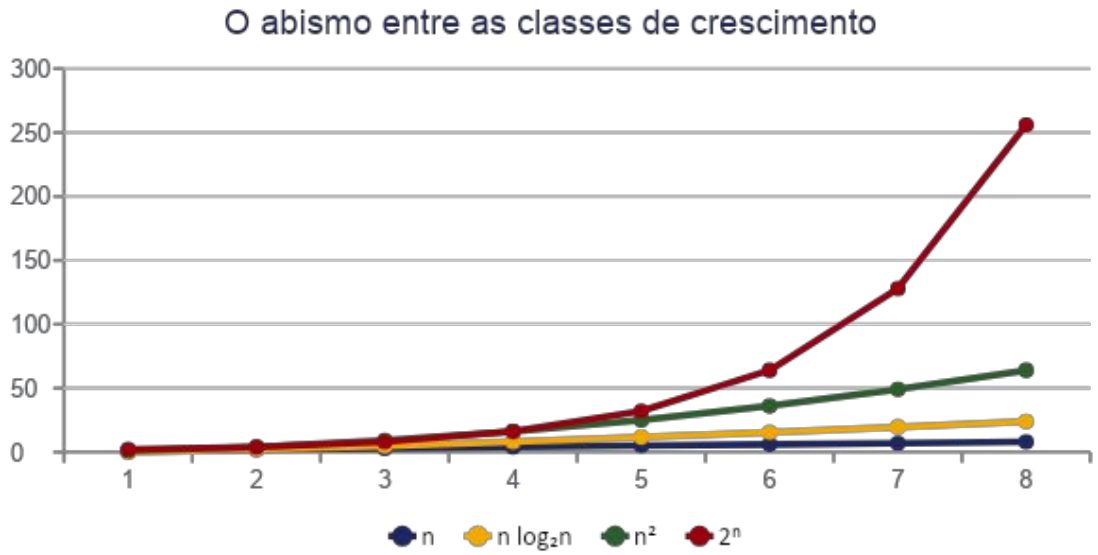
Direto: (3·1+2)+(3·2+2) = 5+8 = 13. Pela fórmula: (3·4+7·2)/2 = 26/2 = 13. ✓

Padrão a reter

Sempre que o custo por iteração é linear em i (a·i+b), o resultado é Θ(n²), Σai domina, porque Σi já é quadrático em n.

Fórmulas fechadas e ordens de crescimento

Somatório	Fórmula fechada	Ordem
$\sum_{i=1}^{n} 1$	n	$\Theta(n)$
$\sum_{i=1}^{n} i$	$n(n+1)/2$	$\Theta(n^2)$
$\sum_{i=1}^{n} i^2$	$n(n+1)(2n+1)/6$	$\Theta(n^3)$
$\sum_{i=0}^{n} c^i$ $(c \neq 1)$	$(c^{n+1}-1)/(c-1)$	$\Theta(c^n)$ se $c>1$
$\sum_{i=1}^{n} 1/i$ (harmônica)	$\approx \ln(n)+\gamma$	$\Theta(\log n)$



Leitura do gráfico

Mesmo em uma escala pequena (n de 1 a 8), 2^n já dispara: é a mesma disparidade que explica por que Hanói (2^n-1) é impraticável para n grande.

Observação

A soma harmônica não tem fórmula fechada elementar, por isso é tratada separadamente na análise de algoritmos (ex.: quicksort médio, union-find com compressão de caminho).

EXERCÍCIO · BLOCO 4

Conte as comparações do bubbleSort

```
Algoritmo bubbleSort(A[1..n])
  para i de 1 até n-1 faça
    para j de 1 até n-i faça
      se A[j] > A[j+1] então
        troca(A[j], A[j+1])
```

Enunciado

Escreva, como um somatório duplo, o número exato de comparações A[j]>A[j+1] no pior caso, e encontre a fórmula fechada em função de n.

Dica

Para cada i fixo, o laço interno j executa (n-i) comparações. O total é $\sum_{i=1}^{n-1}(n-i)$. Faça k=n-i para simplificar.

Por que importa na prática

É exatamente essa contagem que justifica, com rigor, por que ordenar 1 milhão de itens com bubbleSort é inviável: o número de comparações cresce com n^2.

Total de comparações = n(n-1)/2

Resolução

Total = $\sum_{i=1}^{n-1} (n-i)$

$= \sum_{k=1}^{n-1} k \qquad (k = n-i)$

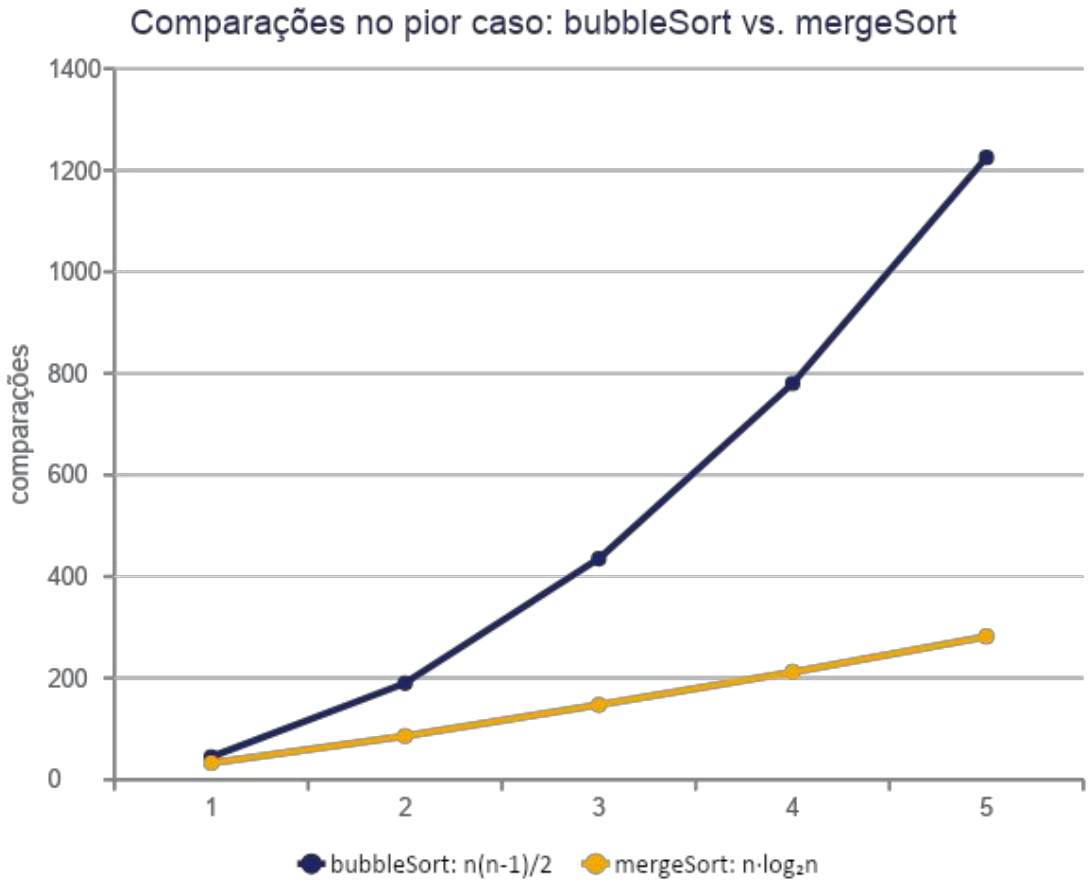
$= (n-1)n/2 = n(n-1)/2.$

Verificação (n=4)

Comparações: 3+2+1=6. Fórmula: 4·3/2=6. ✓

Impacto real do gráfico ao lado

Em n=50, bubbleSort já faz 4x mais comparações que mergeSort. Em n=1.000.000, a diferença é de milhões de vezes, a razão pela qual bibliotecas de ordenação usam algoritmos O(n log n).



O conceito de soma telescópica

$$\sum_{i=1}^{n} (a_i - a_{i-1}) = a_n - a_0$$

Por que os termos do meio cancelam

$(a_1-a_0) + (a_2-a_1) + (a_3-a_2) + \ldots + (a_n-a_{n-1})$

Cada termo $+a_i$ é cancelado pelo termo $-a_i$ do parênteses seguinte (como um telescópio que se retrai). Só sobrevivem $-a_0$ (o primeiro) e a_n (o último).

Prova formal (por indução!)

Base (n=1): $\sum^1 = a_1-a_0$. Passo: assumindo $\sum^n = a_n-a_0$, então $\sum^{n+1} = (a_n-a_0)+(a_{n+1}-a_n) = a_{n+1}-a_0$.

Estratégia de uso

Para resolver um somatório difícil, tente reescrever o termo geral como uma diferença a_i-a_{i-1}. Se conseguir, a soma inteira colapsa para dois termos.

Exemplo clássico: frações parciais

Calcular $\sum_{k=1}^{n} 1/(k(k+1))$.

Passo 1: decompor em frações parciais

$1/(k(k+1)) = 1/k - 1/(k+1)$ (verifique pelo mmc: $(k+1-k)/(k(k+1)) = 1/(k(k+1))$ ✓)

$\sum_{k=1}^{n} (1/k - 1/(k+1)) = 1/1 - 1/(n+1) = n/(n+1)$.

Aplicação real 1: confiabilidade de redes

Essa soma modela a probabilidade acumulada de falha em certos processos de confiabilidade encadeada (cada elo depende do anterior falhar antes).

Aplicação real 2: extrato bancário

O saldo final de uma conta é uma soma telescópica dos lançamentos: $\text{saldo}_n = \text{saldo}_0 + \sum(\text{saldo}_i - \text{saldo}_{i-1})$, só o saldo inicial e o final importam para o total movimentado.

Observação notável

Quando $n \to \infty$, a soma tende a 1, uma série infinita que converge, tornada óbvia pela telescópica.

Reobtendo a soma de Gauss por telescópica

Ideia

Defina $a_i = i(i+1)/2$. Note que:

$a_i - a_{i-1} = i(i+1)/2 - (i-1)i/2 = i \cdot 2/2 = i$.

Logo $\Sigma^n_{i=1}$ i é, ela mesma, telescópica

$\Sigma^n_{i=1} i = \Sigma^n_{i=1} (a_i - a_{i-1}) = a\square - a_0 = n(n+1)/2$.

Chegamos à MESMA fórmula do slide 26, sem indução explícita, direto pela telescópica.

O ponto pedagógico

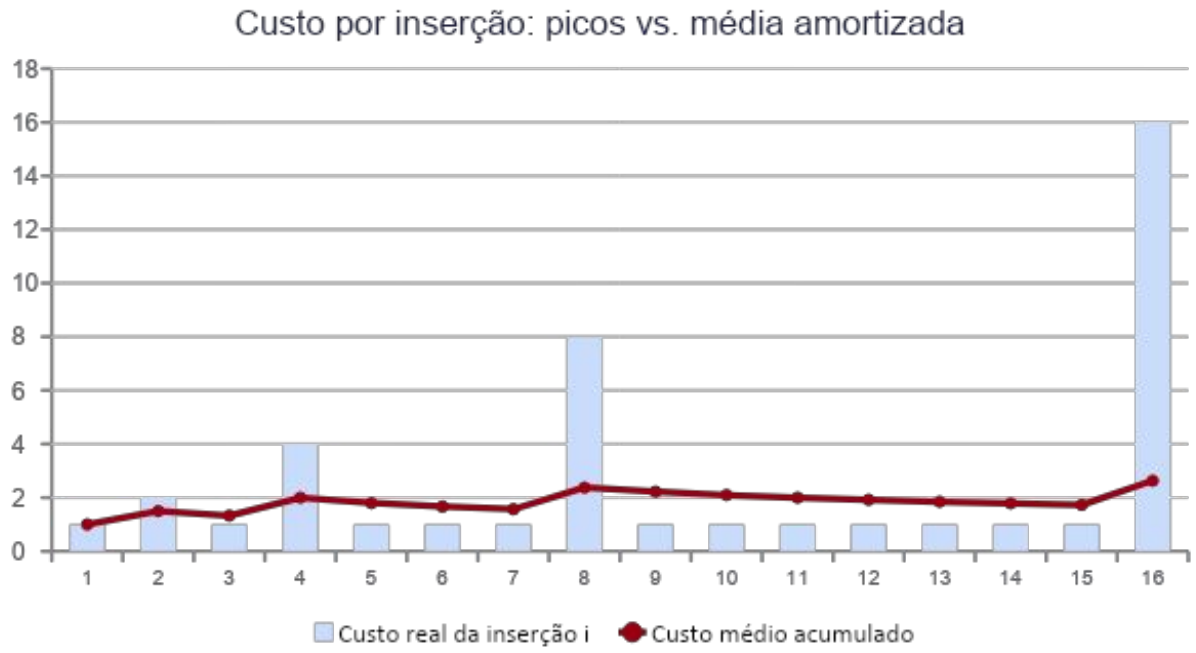
Indução e telescópica não são técnicas concorrentes, a telescópica é um atalho algébrico cuja validade É provada por indução (slide 37).

Quando preferir cada técnica

- Já 'adivinha' a fórmula fechada e só precisa confirmar ⟶ indução direta.
- Não sabe a fórmula, mas consegue reescrever o termo como diferença ⟶ telescópica costuma REVELAR a fórmula fechada, em vez de apenas confirmá-la.

Soma telescópica em análise amortizada

```
Algoritmo inserir(V, x)
  se tamanho(V) = capacidade(V) então
    nova ← alocarVetor(2·capacidade(V))
    copiar todos elementos de V para nova
    V ← nova
  V[tamanho(V)] ← x
```



Contexto real

Assim crescem std::vector (C++), ArrayList (Java) e list (Python) por dentro: quando lotam, dobram e copiam tudo. Custo total após n inserções?

Custo total de cópias (capacidade dobra em 1,2,4,8,...,n)

$\sum_{i=0}^{k} 2^i$ onde $2^k \leq n < 2^{k+1} = 2^{k+1}-1 < 2n$.

Conclusão (amortização)

Custo total < 2n para n inserções $\Longrightarrow$ custo amortizado O(1) por inserção, mesmo que uma inserção isolada custe O(n) no pior caso. É a mesma soma geométrica telescópica do slide 38.

EXERCÍCIO · SÍNTESE

Combine indução, somatório e telescópica

Enunciado

Um algoritmo tem $T(1)=1$ e $T(n)=T(n-1)+n$ para $n \geq 2$, a mesma recorrência de insertionSort (slide 19) no pior caso.
(a) Escreva $T(n)$ como um somatório. (b) Encontre a fórmula fechada. (c) Prove-a por indução.

Dica

Para (a), 'desenrole' a recorrência: $T(n)=T(1)+\sum_{i=2}^{n} i$. Para (b), use Gauss. Para (c), reaproveite o roteiro do slide 26.

Por que este exercício é o mais importante da aula

$T(n)=T(n-1)+n$ é exatamente a recorrência de insertionSort (Parte I) e de selectionSort no pior caso. Resolvê-la, somatório + fórmula fechada + prova por indução, é o passo central de toda análise de complexidade que vocês farão no resto da disciplina.

$T(n) = T(1) + \sum_{i=2}^{n} i = n(n+1)/2$

(a) Desenrolando a recorrência

$T(n) = T(n-1)+n = \ldots = T(1)+\sum_{i=2}^{n} i = 1+\sum_{i=2}^{n} i = \sum_{i=1}^{n} i$.

(b) Fórmula fechada

$T(n) = \sum_{i=1}^{n} i = n(n+1)/2$ (Gauss, slide 24).

(c) Prova formal por indução de que T(n) = n(n+1)/2

Base (n=1): $T(1)=1=1\cdot 2/2$. Vale.

Hipótese ($n \geq 1$): $T(n)=n(n+1)/2$.

Passo: mostrar que $T(n+1)=(n+1)(n+2)/2$:

$T(n+1) = T(n)+(n+1) = n(n+1)/2+(n+1) = (n+1)(n+2)/2$.

Conclusão da aula

$T(n) = \Theta(n^2)$, usamos somatório para ENCONTRAR a fórmula e indução para PROVÁ-la rigorosamente. As duas ferramentas são complementares, e a prova de corretude que abriu a aula usa exatamente essa mesma lógica de base+hipótese+passo.

SÍNTESE

O que levar desta aula

- Corretude de algoritmos (recursivos e iterativos) é sempre provada com a mesma ferramenta: indução, base, hipótese e passo, seja no tamanho da entrada, seja no índice da iteração.

- Testes e provas de corretude se complementam: testes acham bugs comuns, provas garantem 'para todo n', a lição de Ariane 5, Therac-25 e Knight Capital.

- Somatório Σ tem definição recursiva e propriedades (linearidade, fator constante, separação) que permitem manipular somas sem calcular termo a termo.

- Fórmulas fechadas (Gauss, quadrática, geométrica) e a soma telescópica transformam recorrências e somatórios difíceis em expressões diretas, a base de toda análise de complexidade.

- Na prática: para resolver T(n), desenrole-a em somatório, encontre a fórmula fechada (direto ou via telescópica), e prove-a formalmente por indução.

Questões?